

VCGen

SeaBMC Style

Idea

SeaBMC-style VCGen compiles control-dependence into data-dependence.

After preprocessing:

- no block variables are needed
- no separate CFG constraints are emitted
- phi nodes become ordinary `ite` definitions
- cone-of-influence reduction is just data-flow slicing

Gated SSA

A gamma node is a value-level merge:

$$x := \gamma(g, x_t, x_f)$$

It means:

$$x := \text{ite}(g, x_t, x_f)$$

Unlike a phi node, gamma mentions a Boolean program expression, not a predecessor block name.

SSA Diamond

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  goto join

right:
  a_right := y + 1
  goto join

join:
  a := phi [left: a_left, right: a_right]
```

GSSA Diamond

The branch condition c selects the incoming region.

```
join:  
  a := gamma(c, a_left, a_right)  
  ok := a > 0  
  goto exit
```

After desugaring:

```
join:  
  a := ite(c, a_left, a_right)  
  ok := a > 0  
  goto exit
```

Flat VC

After phi elimination:

$$\text{VCGen_SeaBMC}(P) = \text{DEF} \wedge \text{ASSUME} \wedge \neg \text{ASSERT}$$

For the diamond:

$$\text{DEF} = c = (x < y) \wedge a_{\text{left}} = x + 1 \wedge a_{\text{right}} = y + 1 \wedge a = \text{ite}(c, a_{\text{left}}, a_{\text{right}}) \wedge \text{ok} = (a > 0)$$

Gate Construction

For each block b , compute a Boolean expression $\text{gate}(b)$:

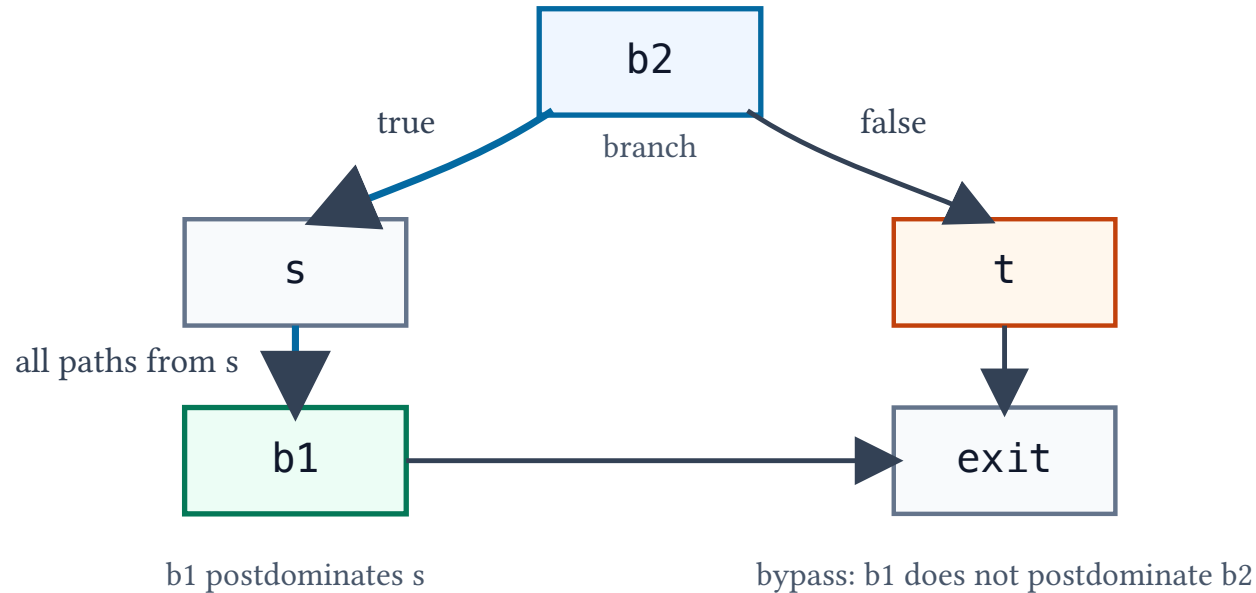
$$\text{gate}(\text{entry}) = \top$$

$$\text{gate}(b) = \bigvee_{c \in \text{ctrl}(b)} (\text{gate}(c) \wedge \text{orient}(c, b))$$

$\text{ctrl}(b)$ contains controlling branch blocks. $\text{orient}(c, b)$ is the branch condition or its negation, depending on which successor reaches b .

Control Dependence

Control dependence: b1 depends on the branch at b2



A block is control-dependent on a branch when one successor forces a later visit to the block, while another successor can bypass it.

Materialized Gates

Expanding every gate at every use can duplicate shared control structure.

Materialized gates introduce ordinary SSA Boolean definitions:

$$\begin{aligned} G_{\text{entry}} &:= \top \\ G_b &:= \bigvee_{c \in \text{ctrl}(b)} (G_c \wedge \text{orient}(c, b)) \\ x &:= \gamma(G_p, v_p, v_q) \end{aligned}$$

The VC remains CFG-free; gates are just definitions.

Shared Gate Example

```
gate_a := p
gate_b := q
gate_c := (gate_a and r) or (gate_b and s)
gate_d := (gate_b and t) or (gate_c and u)
gate_e := (gate_c and v) or (gate_d and w)

join1:
  x := ite(gate_d, x_d, ite(gate_e, x_e, x_other))

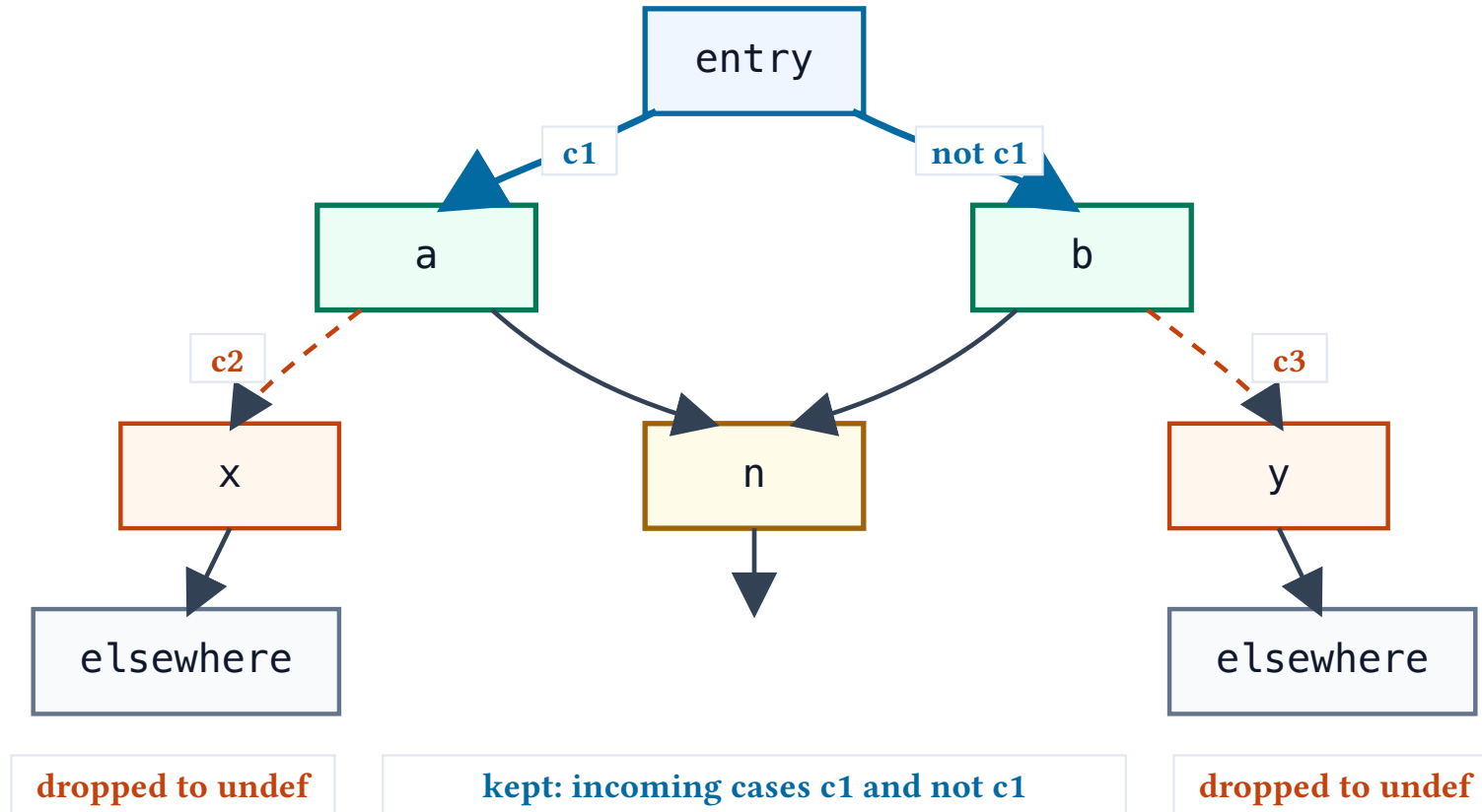
join2:
  y := ite(gate_d, y_d, ite(gate_e, y_e, y_other))
```

The shared control-dependence DAG is represented once.

Thin GSSA

Thin GSSA uses only direct control-dependence controllers.

Thin GSSA keeps the incoming cases to n and drops bypass cases to undef



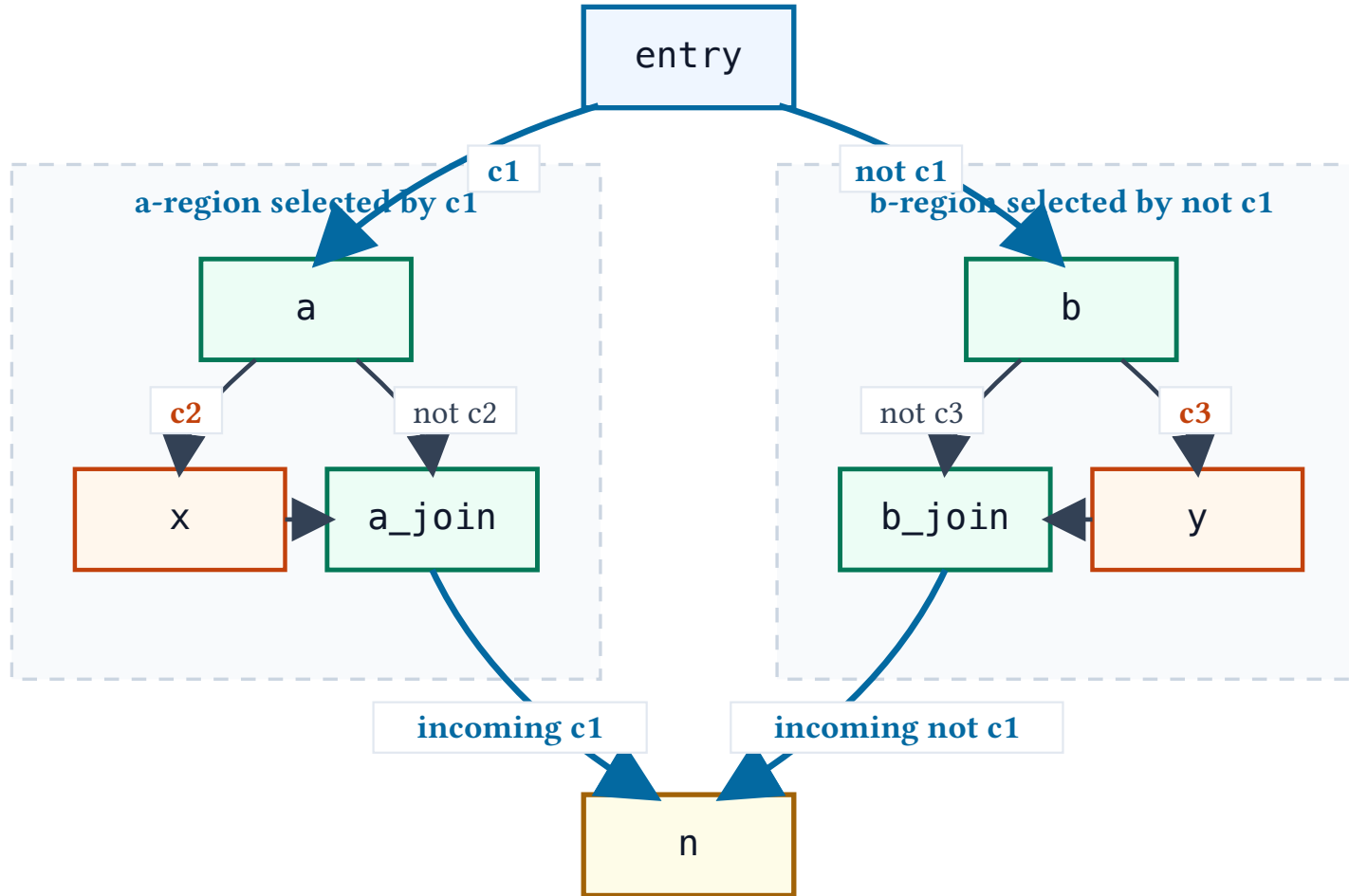
For a direct controller c of block b :

$$K_{c,b} = G_c \wedge \text{orient}(c, b)$$

The G_c factor prevents unreachable branch conditions from selecting values.

Thin Example CFG

CFG for the Thin GSSA example



The outer branch selects the region feeding n ; local branches only compute values inside each region.

Thin Example: Final Form

```
a_join:
  v_a := ite(c2, v_x, v_a0)

b_join:
  v_b := ite(c3, v_y, v_b0)

n:
  v := ite(c1, v_a, v_b)
```

The final phi at n depends only on the region selector c_1 , not on the local branch choices c_2 and c_3 .

Cone Of Influence

Once control is data:

1. Start from the exit assume and assert.
2. Keep each used variable definition.
3. Add the variables used by that definition.
4. Keep gates only when retained ITEs use them.
5. Drop every unmarked definition.

No graph reachability rule is needed for pruning.

COI Example

```
n:  
  v := ite(c1, v_a, v_b)  
  junk := expensive(v_x, v_y)  
  ok := v > 0  
  
exit:  
  assume true  
  assert ok
```

junk is dropped because it is not used by `ok`, the exit assumption, or a retained gate.